

Task 1

1.

```
class Linear(Module):
    def __init__(self, in_features, out_features) -> None:
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features

        # Initialize weights and biases
        init_factor = 0.01
        self.W = Parameter(np.random.randn(in_features, out_features) * init_factor)
        self.b = Parameter(np.zeros((1, out_features)))

        # Cache for backward pass
        self.x = None

    def forward(self, x): #FC
        # 學生實作部分: return output of linear layer
        self.x = x #輸入x把其存起來
        x = x @ self.W.data + self.b.data #以內積形式運算並用 .data存取所需值
        return x

    def backward(self, dy):
        # 學生實作部分: return gradient w.r.t. input and compute gradients for weights and biases
        self.W.grad = self.x.T @ dy #對W作微分, 即dE/dW = X*(dE/dY)
        self.b.grad = np.sum(dy, axis=0, keepdims=True) #對bias做微分
        dx = dy @ self.W.data.T #dE/dX = (dE/dY)*(dY/dX) ;其中(dY/dX)其實就是權重值
        return dx
```

```
class ReLU(Module):
    def __init__(self) -> None:
        super().__init__()
        self.x = None

    def forward(self, x):
        # 學生實作部分: return output of ReLU activation
        self.x = x #用於稍後BP
        out = np.maximum(0, x)
        return out

    def backward(self, dy):
        # 學生實作部分: return gradient w.r.t. input
        dx = dy.copy()
        dx[self.x <= 0] = 0 #當f(x)為0微分一樣0
        return dx

    def __repr__(self) -> str:
        return f'{self.__class__.__name__}()'
```

```

class Sigmoid(Module):
    def __init__(self) -> None:
        super().__init__()
        self.y = None

    def forward(self, x):
        # 學生實作部分: return output of Sigmoid activation
        self.x = x
        self.y = 1 / (1 + np.exp(-x)) #  $y = 1/(1 + e^{-x})$ 
        return self.y

    def backward(self, dy):
        # 學生實作部分: return gradient w.r.t. input
        return dy * self.y * (1 - self.y) #即sigmoid公式做微分推導至最簡式子

    def __repr__(self) -> str:
        return f'{self.__class__.__name__}()'

```

```

class Softmax(Module):
    def __init__(self) -> None:
        super().__init__()
        self.y = None

    def forward(self, x):
        # 學生實作部分: return output of Softmax activation
        self.x = x
        # Subtract the maximum value for numerical stability
        exp_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
        self.y = exp_x / np.sum(exp_x, axis=-1, keepdims=True)
        return self.y

    def backward(self, dy):
        # 這邊我們設定同學們會將Softmax和Cross-Entropy Loss一起使用
        # 因此backward pass根據講義上的說明使用簡化的版本在Cross-Entropy Loss的部分完成
        # 若同學的softmax沒有和Cross-Entropy Loss一起使用, 請自行實作完整的backward pass
        return dy

    def __repr__(self) -> str:
        return f'{self.__class__.__name__}()'

```

2. epoch=8 ; lr=0.001; 驗證採用 10-Fold-Cross-Validation

3.

```
class MLP(Module):
    def __init__(self) -> None:
        # 學生實作部分: design your Model architecture here
        super().__init__()
        self.fc1 = Linear(28 * 28, 64) # Input layer to first hidden layer
        self.relu1 = ReLU()
        self.fc2 = Linear(64, 32) # First hidden layer to second hidden layer
        self.relu2 = ReLU()
        self.fc3 = Linear(32, 10) # Second hidden layer to output layer (should be 10 for MNIST classes)
        self.softmax = Softmax() # Softmax activation for output

    def forward(self, x):
        # 學生實作部分: compute forward pass through your model
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        x = self.softmax(x)
        return x

    def backward(self, dy):
        # 學生實作部分: compute backward pass through your model
        dy = self.softmax.backward(dy)
        dy = self.fc3.backward(dy)
        dy = self.relu2.backward(dy)
        dy = self.fc2.backward(dy)
        dy = self.relu1.backward(dy)
        dy = self.fc1.backward(dy)
        return dy

    def parameters(self):
        return self.fc1.parameters() + self.fc2.parameters() + self.fc3.parameters()
```

```
100%|██████████| 54000/54000 [00:27<00:00, 1939.95it/s]
epoch 0: train_loss = 0.43240533888187277, train_acc = 0.8818518518518519
100%|██████████| 6000/6000 [00:00<00:00, 9175.90it/s]
epoch 0: valid_loss = 0.5386841989303203, valid_acc = 0.8841666666666667
```

```
100%|██████████| 54000/54000 [00:28<00:00, 1897.92it/s]
epoch 1: train_loss = 0.3291542515506896, train_acc = 0.9208333333333333
100%|██████████| 6000/6000 [00:00<00:00, 8959.89it/s]
epoch 1: valid_loss = 0.29708915492048416, valid_acc = 0.9221666666666667
```

```
100%|██████████| 54000/54000 [00:28<00:00, 1909.30it/s]
epoch 2: train_loss = 0.2920680435396002, train_acc = 0.9318518518518518
100%|██████████| 6000/6000 [00:00<00:00, 9369.14it/s]
epoch 2: valid_loss = 0.28538683749501664, valid_acc = 0.9296666666666666
```

```
100%|██████████| 54000/54000 [00:27<00:00, 1928.65it/s]
epoch 3: train_loss = 0.2665090049756341, train_acc = 0.9375925925925926
100%|██████████| 6000/6000 [00:00<00:00, 6235.71it/s]
epoch 3: valid_loss = 0.32157616656830323, valid_acc = 0.932
```

```
100%|██████████| 54000/54000 [00:28<00:00, 1906.42it/s]
epoch 4: train_loss = 0.28105362663080125, train_acc = 0.9376111111111111
100%|██████████| 6000/6000 [00:00<00:00, 6534.34it/s]
epoch 4: valid_loss = 0.31072849099232347, valid_acc = 0.9411666666666667
```

```
100%|██████████| 54000/54000 [00:27<00:00, 1931.25it/s]
epoch 5: train_loss = 0.25239607996420205, train_acc = 0.942962962962963
100%|██████████| 6000/6000 [00:01<00:00, 5532.66it/s]
epoch 5: valid_loss = 0.26167399883580184, valid_acc = 0.9368333333333333
```

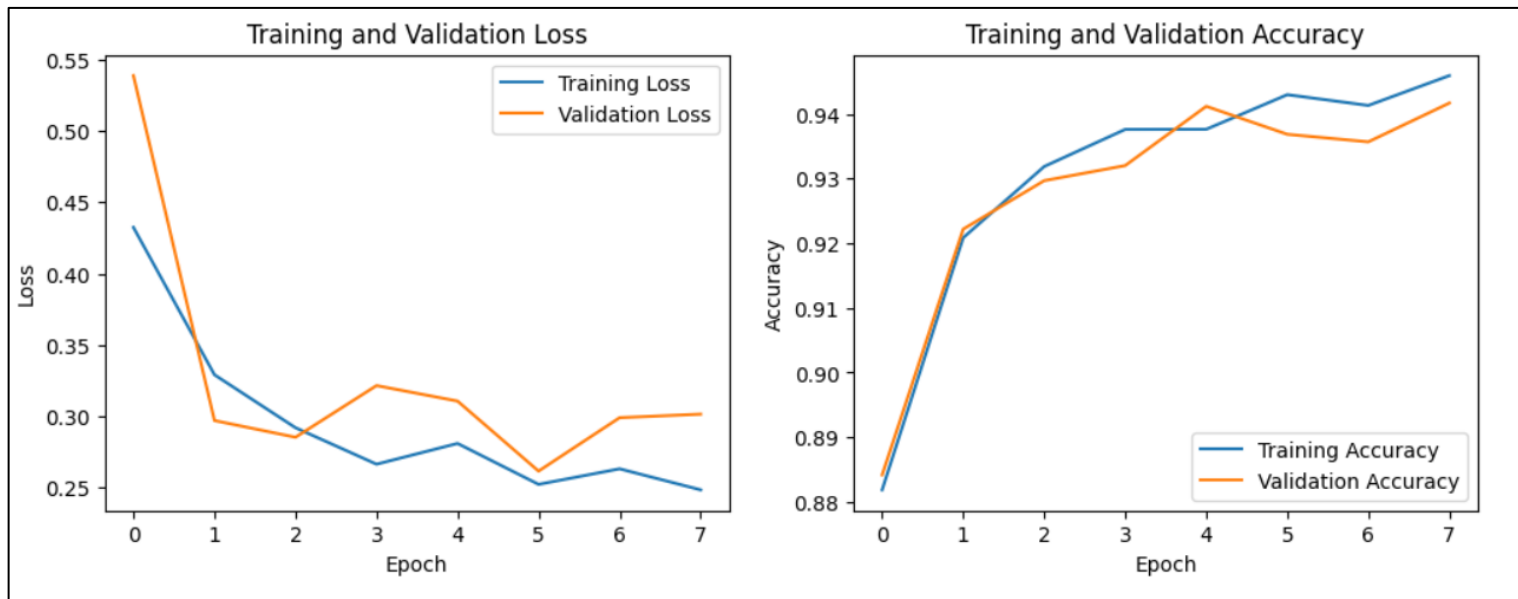
```
100%|██████████| 54000/54000 [00:27<00:00, 1928.65it/s]
epoch 6: train_loss = 0.26328392266380796, train_acc = 0.9412962962962963
100%|██████████| 6000/6000 [00:00<00:00, 9092.11it/s]
epoch 6: valid_loss = 0.29909325152665317, valid_acc = 0.9356666666666666
```

```
100%|██████████| 54000/54000 [00:28<00:00, 1926.12it/s]
epoch 7: train_loss = 0.24855732586908413, train_acc = 0.9459074074074074
100%|██████████| 6000/6000 [00:00<00:00, 9546.86it/s]epoch 7: valid_loss = 0.30156470228737625, valid_acc = 0.9416666666666667
```

```
# Using test_images and test_labels to do the final test
test_loss, test_acc = evaluate(net, testldr)
print(f"test_loss = {test_loss}, test_acc = {test_acc}")
```

```
100%|██████████| 10000/10000 [00:01<00:00, 6551.13it/s]test_loss = 0.29901393167699325, test_acc = 0.9411
```

4.



Task2

1.

```
class BasicBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super(BasicBlock, self).__init__()
        # 學生實作部分: Define the two convolutional layers and the shortcut connection
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True) # Add ReLU here
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(out_channels) # Add BatchNorm to shortcut
            )

    def forward(self, x):
        # 學生實作部分: Define the forward pass using convolutional layers and the shortcut connection
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
        out = self.conv2(out)
        out = self.bn2(out)
        out += self.shortcut(x) # Add shortcut connection
        out = self.relu(out) # Apply ReLU after adding shortcut
        return out
```

```
class ResNet18(nn.Module):
    def __init__(self, num_classes=1000): # Default num_classes to 1000 for ImageNet-like structure
        super(ResNet18, self).__init__()
        # 學生實作部分: Define the ResNet-18 architecture using BasicBlock
        self.in_channels = 64
        self.conv1 = nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

        # ResNet layers
        self.layer1 = self._make_layer(BasicBlock, 64, 2, stride=1)
        self.layer2 = self._make_layer(BasicBlock, 128, 2, stride=2)
        self.layer3 = self._make_layer(BasicBlock, 256, 2, stride=2)
        self.layer4 = self._make_layer(BasicBlock, 512, 2, stride=2)
        self.avgpool = nn.AdaptiveAvgPool2d((1,1))
        self.fc = nn.Linear(512, num_classes)
        self.softmax = nn.Softmax(dim=1) # Added Softmax as CrossEntropyLoss might not be used alone

    def _make_layer(self, block, out_channels, num_blocks, stride):
        # 學生實作部分: Define make_layer function to create layers of blocks
        strides = [stride] + [1] * (num_blocks - 1)
        layers = []
        for stride in strides:
            layers.append(block(self.in_channels, out_channels, stride))
            self.in_channels = out_channels
        return nn.Sequential(*layers)

    def forward(self, x):
        # 學生實作部分: Define the forward pass of ResNet-18
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
        out = self.maxpool(out) # Added maxpool
        out = self.layer1(out)
        out = self.layer2(out)
        out = self.layer3(out)
        out = self.layer4(out)
        out = self.avgpool(out)
        out = out.view(out.size(0), -1)
        out = self.fc(out)
        out = self.softmax(out) # Added softmax
        return out
```

2.

Layer (type)	Output Shape	Param #
-----	-----	-----
Conv2d-1	[-1, 64, 112, 112]	9,408
BatchNorm2d-2	[-1, 64, 112, 112]	128
ReLU-3	[-1, 64, 112, 112]	0
MaxPool2d-4	[-1, 64, 56, 56]	0
Conv2d-5	[-1, 64, 56, 56]	36,864
BatchNorm2d-6	[-1, 64, 56, 56]	128
ReLU-7	[-1, 64, 56, 56]	0
Conv2d-8	[-1, 64, 56, 56]	36,864
BatchNorm2d-9	[-1, 64, 56, 56]	128
ReLU-10	[-1, 64, 56, 56]	0
BasicBlock-11	[-1, 64, 56, 56]	0
Conv2d-12	[-1, 64, 56, 56]	36,864
BatchNorm2d-13	[-1, 64, 56, 56]	128
ReLU-14	[-1, 64, 56, 56]	0
Conv2d-15	[-1, 64, 56, 56]	36,864
BatchNorm2d-16	[-1, 64, 56, 56]	128
ReLU-17	[-1, 64, 56, 56]	0
BasicBlock-18	[-1, 64, 56, 56]	0
Conv2d-19	[-1, 128, 28, 28]	73,728
BatchNorm2d-20	[-1, 128, 28, 28]	256
ReLU-21	[-1, 128, 28, 28]	0
Conv2d-22	[-1, 128, 28, 28]	147,456
BatchNorm2d-23	[-1, 128, 28, 28]	256
Conv2d-24	[-1, 128, 28, 28]	8,192
BatchNorm2d-25	[-1, 128, 28, 28]	256
ReLU-26	[-1, 128, 28, 28]	0
BasicBlock-27	[-1, 128, 28, 28]	0
Conv2d-28	[-1, 128, 28, 28]	147,456
BatchNorm2d-29	[-1, 128, 28, 28]	256
ReLU-30	[-1, 128, 28, 28]	0
Conv2d-31	[-1, 128, 28, 28]	147,456
BatchNorm2d-32	[-1, 128, 28, 28]	256
ReLU-33	[-1, 128, 28, 28]	0
BasicBlock-34	[-1, 128, 28, 28]	0

Conv2d-35	[-1, 256, 14, 14]	294,912
BatchNorm2d-36	[-1, 256, 14, 14]	512
ReLU-37	[-1, 256, 14, 14]	0
Conv2d-38	[-1, 256, 14, 14]	589,824
BatchNorm2d-39	[-1, 256, 14, 14]	512
Conv2d-40	[-1, 256, 14, 14]	32,768
BatchNorm2d-41	[-1, 256, 14, 14]	512
ReLU-42	[-1, 256, 14, 14]	0
BasicBlock-43	[-1, 256, 14, 14]	0
Conv2d-44	[-1, 256, 14, 14]	589,824
BatchNorm2d-45	[-1, 256, 14, 14]	512
ReLU-46	[-1, 256, 14, 14]	0
Conv2d-47	[-1, 256, 14, 14]	589,824
BatchNorm2d-48	[-1, 256, 14, 14]	512
ReLU-49	[-1, 256, 14, 14]	0
BasicBlock-50	[-1, 256, 14, 14]	0
Conv2d-51	[-1, 512, 7, 7]	1,179,648
BatchNorm2d-52	[-1, 512, 7, 7]	1,024
ReLU-53	[-1, 512, 7, 7]	0
Conv2d-54	[-1, 512, 7, 7]	2,359,296
BatchNorm2d-55	[-1, 512, 7, 7]	1,024
Conv2d-56	[-1, 512, 7, 7]	131,072
BatchNorm2d-57	[-1, 512, 7, 7]	1,024
ReLU-58	[-1, 512, 7, 7]	0
BasicBlock-59	[-1, 512, 7, 7]	0
Conv2d-60	[-1, 512, 7, 7]	2,359,296
BatchNorm2d-61	[-1, 512, 7, 7]	1,024
ReLU-62	[-1, 512, 7, 7]	0
Conv2d-63	[-1, 512, 7, 7]	2,359,296
BatchNorm2d-64	[-1, 512, 7, 7]	1,024
ReLU-65	[-1, 512, 7, 7]	0
BasicBlock-66	[-1, 512, 7, 7]	0
AdaptiveAvgPool2d-67	[-1, 512, 1, 1]	0
Linear-68	[-1, 10]	5,130
Softmax-69	[-1, 10]	0

```
Total params: 11,181,642
Trainable params: 11,181,642
Non-trainable params: 0
```

```
Input size (MB): 0.57
Forward/backward pass size (MB): 62.79
Params size (MB): 42.65
Estimated Total Size (MB): 106.01
```

```
[INFO] Register count_convNd() for <class 'torch.nn.modules.conv.Conv2d'>.
[INFO] Register count_normalization() for <class 'torch.nn.modules.batchnorm.BatchNorm2d'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.activation.ReLU'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.pooling.MaxPool2d'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.container.Sequential'>.
[INFO] Register count_adap_avgpool() for <class 'torch.nn.modules.pooling.AdaptiveAvgPool2d'>.
[INFO] Register count_linear() for <class 'torch.nn.modules.linear.Linear'>.
[INFO] Register count_softmax() for <class 'torch.nn.modules.activation.Softmax'>.
FLOPs: 37.22 MFLOPs
Params: 11.18 M
```

3.

Batch_size=256

EPOCH = 100

pre_epoch = 0

lr = 0.01

4.

Epoch [1/100]	Train Loss: 2.1580	Train Acc: 29.03%	Val Loss: 2.1086	Val Acc: 34.64%
Epoch [2/100]	Train Loss: 2.0691	Train Acc: 38.51%	Val Loss: 2.0557	Val Acc: 40.00%
Epoch [3/100]	Train Loss: 2.0199	Train Acc: 43.50%	Val Loss: 2.0531	Val Acc: 40.48%
Epoch [4/100]	Train Loss: 1.9876	Train Acc: 46.90%	Val Loss: 1.9931	Val Acc: 46.22%
Epoch [5/100]	Train Loss: 1.9554	Train Acc: 50.15%	Val Loss: 1.9651	Val Acc: 48.78%
Epoch [6/100]	Train Loss: 1.9421	Train Acc: 51.52%	Val Loss: 1.9499	Val Acc: 50.62%
Epoch [7/100]	Train Loss: 1.9185	Train Acc: 53.94%	Val Loss: 1.9268	Val Acc: 53.12%
Epoch [8/100]	Train Loss: 1.9114	Train Acc: 54.64%	Val Loss: 1.9452	Val Acc: 51.08%
Epoch [9/100]	Train Loss: 1.8914	Train Acc: 56.61%	Val Loss: 1.9305	Val Acc: 52.78%
Epoch [10/100]	Train Loss: 1.8795	Train Acc: 57.81%	Val Loss: 1.9159	Val Acc: 54.14%
Epoch [11/100]	Train Loss: 1.8665	Train Acc: 59.19%	Val Loss: 1.9044	Val Acc: 55.46%
Epoch [12/100]	Train Loss: 1.8575	Train Acc: 60.03%	Val Loss: 1.8923	Val Acc: 56.56%
Epoch [13/100]	Train Loss: 1.8472	Train Acc: 61.19%	Val Loss: 1.8771	Val Acc: 57.96%
Epoch [14/100]	Train Loss: 1.8403	Train Acc: 61.79%	Val Loss: 1.8931	Val Acc: 56.20%
Epoch [15/100]	Train Loss: 1.8382	Train Acc: 61.95%	Val Loss: 1.8695	Val Acc: 58.44%
Epoch [16/100]	Train Loss: 1.8282	Train Acc: 63.04%	Val Loss: 1.8709	Val Acc: 58.52%
Epoch [17/100]	Train Loss: 1.8235	Train Acc: 63.55%	Val Loss: 1.8492	Val Acc: 60.92%
Epoch [18/100]	Train Loss: 1.8134	Train Acc: 64.55%	Val Loss: 1.8572	Val Acc: 59.90%
Epoch [19/100]	Train Loss: 1.8095	Train Acc: 65.02%	Val Loss: 1.8436	Val Acc: 61.38%
Epoch [20/100]	Train Loss: 1.7999	Train Acc: 65.89%	Val Loss: 1.8441	Val Acc: 61.34%
Epoch [21/100]	Train Loss: 1.7993	Train Acc: 66.02%	Val Loss: 1.8501	Val Acc: 60.56%
Epoch [22/100]	Train Loss: 1.7961	Train Acc: 66.30%	Val Loss: 1.8413	Val Acc: 61.62%
Epoch [23/100]	Train Loss: 1.7862	Train Acc: 67.38%	Val Loss: 1.8476	Val Acc: 60.78%
Epoch [24/100]	Train Loss: 1.7859	Train Acc: 67.41%	Val Loss: 1.8282	Val Acc: 63.18%
Epoch [25/100]	Train Loss: 1.7845	Train Acc: 67.53%	Val Loss: 1.8296	Val Acc: 62.56%
Epoch [26/100]	Train Loss: 1.7764	Train Acc: 68.32%	Val Loss: 1.8196	Val Acc: 63.74%
Epoch [27/100]	Train Loss: 1.7725	Train Acc: 68.75%	Val Loss: 1.8212	Val Acc: 63.72%
Epoch [28/100]	Train Loss: 1.7682	Train Acc: 69.10%	Val Loss: 1.8105	Val Acc: 64.68%
Epoch [29/100]	Train Loss: 1.7681	Train Acc: 69.15%	Val Loss: 1.8176	Val Acc: 63.92%
Epoch [30/100]	Train Loss: 1.7619	Train Acc: 69.82%	Val Loss: 1.8230	Val Acc: 63.76%
Epoch [31/100]	Train Loss: 1.7618	Train Acc: 69.77%	Val Loss: 1.8180	Val Acc: 64.10%
Epoch [32/100]	Train Loss: 1.7558	Train Acc: 70.42%	Val Loss: 1.8059	Val Acc: 65.22%
Epoch [33/100]	Train Loss: 1.7546	Train Acc: 70.51%	Val Loss: 1.8119	Val Acc: 64.66%
Epoch [34/100]	Train Loss: 1.7465	Train Acc: 71.37%	Val Loss: 1.8011	Val Acc: 65.44%
Epoch [35/100]	Train Loss: 1.7449	Train Acc: 71.49%	Val Loss: 1.8092	Val Acc: 64.88%
Epoch [36/100]	Train Loss: 1.7460	Train Acc: 71.37%	Val Loss: 1.8191	Val Acc: 63.94%
Epoch [37/100]	Train Loss: 1.7370	Train Acc: 72.27%	Val Loss: 1.8005	Val Acc: 65.66%
Epoch [38/100]	Train Loss: 1.7379	Train Acc: 72.18%	Val Loss: 1.8110	Val Acc: 64.84%
Epoch [39/100]	Train Loss: 1.7360	Train Acc: 72.36%	Val Loss: 1.7952	Val Acc: 66.44%
Epoch [40/100]	Train Loss: 1.7274	Train Acc: 73.23%	Val Loss: 1.8006	Val Acc: 65.66%
Epoch [41/100]	Train Loss: 1.7282	Train Acc: 73.21%	Val Loss: 1.7941	Val Acc: 66.26%
Epoch [42/100]	Train Loss: 1.7277	Train Acc: 73.27%	Val Loss: 1.8041	Val Acc: 65.56%
Epoch [43/100]	Train Loss: 1.7305	Train Acc: 72.91%	Val Loss: 1.7971	Val Acc: 66.12%
Epoch [44/100]	Train Loss: 1.7280	Train Acc: 73.20%	Val Loss: 1.7956	Val Acc: 66.32%
Epoch [45/100]	Train Loss: 1.7270	Train Acc: 73.26%	Val Loss: 1.7970	Val Acc: 65.90%
Epoch [46/100]	Train Loss: 1.7177	Train Acc: 74.28%	Val Loss: 1.7842	Val Acc: 67.48%
Epoch [47/100]	Train Loss: 1.7162	Train Acc: 74.39%	Val Loss: 1.7919	Val Acc: 66.56%
Epoch [48/100]	Train Loss: 1.7141	Train Acc: 74.60%	Val Loss: 1.7811	Val Acc: 67.78%
Epoch [49/100]	Train Loss: 1.7083	Train Acc: 75.20%	Val Loss: 1.7859	Val Acc: 67.20%
Epoch [50/100]	Train Loss: 1.7110	Train Acc: 74.96%	Val Loss: 1.7897	Val Acc: 66.78%
Epoch [51/100]	Train Loss: 1.7074	Train Acc: 75.23%	Val Loss: 1.7794	Val Acc: 67.76%
Epoch [52/100]	Train Loss: 1.7027	Train Acc: 75.75%	Val Loss: 1.7774	Val Acc: 67.90%
Epoch [53/100]	Train Loss: 1.7030	Train Acc: 75.75%	Val Loss: 1.7754	Val Acc: 68.12%
Epoch [54/100]	Train Loss: 1.7007	Train Acc: 75.96%	Val Loss: 1.7663	Val Acc: 69.18%
Epoch [55/100]	Train Loss: 1.6972	Train Acc: 76.30%	Val Loss: 1.7801	Val Acc: 67.66%
Epoch [56/100]	Train Loss: 1.6995	Train Acc: 76.16%	Val Loss: 1.7781	Val Acc: 67.78%
Epoch [57/100]	Train Loss: 1.6949	Train Acc: 76.58%	Val Loss: 1.7716	Val Acc: 68.60%
Epoch [58/100]	Train Loss: 1.6900	Train Acc: 77.11%	Val Loss: 1.7875	Val Acc: 67.10%
Epoch [59/100]	Train Loss: 1.6944	Train Acc: 76.59%	Val Loss: 1.7853	Val Acc: 67.04%
Epoch [60/100]	Train Loss: 1.6900	Train Acc: 77.09%	Val Loss: 1.7655	Val Acc: 69.28%
Epoch [61/100]	Train Loss: 1.6916	Train Acc: 76.83%	Val Loss: 1.7645	Val Acc: 69.34%
Epoch [62/100]	Train Loss: 1.6876	Train Acc: 77.37%	Val Loss: 1.7771	Val Acc: 68.20%
Epoch [63/100]	Train Loss: 1.6892	Train Acc: 77.21%	Val Loss: 1.7912	Val Acc: 66.82%
Epoch [64/100]	Train Loss: 1.6861	Train Acc: 77.49%	Val Loss: 1.7729	Val Acc: 68.56%
Epoch [65/100]	Train Loss: 1.6861	Train Acc: 77.45%	Val Loss: 1.7624	Val Acc: 69.52%
Epoch [66/100]	Train Loss: 1.6778	Train Acc: 78.35%	Val Loss: 1.7523	Val Acc: 70.64%
Epoch [67/100]	Train Loss: 1.6765	Train Acc: 78.44%	Val Loss: 1.7669	Val Acc: 69.06%
Epoch [68/100]	Train Loss: 1.6804	Train Acc: 78.00%	Val Loss: 1.7623	Val Acc: 69.26%
Epoch [69/100]	Train Loss: 1.6757	Train Acc: 78.51%	Val Loss: 1.7722	Val Acc: 68.56%
Epoch [70/100]	Train Loss: 1.6764	Train Acc: 78.47%	Val Loss: 1.7564	Val Acc: 70.22%
Epoch [71/100]	Train Loss: 1.6710	Train Acc: 78.92%	Val Loss: 1.7849	Val Acc: 67.32%
Epoch [72/100]	Train Loss: 1.6743	Train Acc: 78.67%	Val Loss: 1.7774	Val Acc: 68.14%
Epoch [73/100]	Train Loss: 1.6722	Train Acc: 78.86%	Val Loss: 1.7638	Val Acc: 69.42%
Epoch [74/100]	Train Loss: 1.6696	Train Acc: 79.10%	Val Loss: 1.7653	Val Acc: 69.18%
Epoch [75/100]	Train Loss: 1.6698	Train Acc: 79.14%	Val Loss: 1.7740	Val Acc: 68.36%
Epoch [76/100]	Train Loss: 1.6695	Train Acc: 79.12%	Val Loss: 1.7646	Val Acc: 69.18%

```
Epoch [77/100] Train Loss: 1.6676 | Train Acc: 79.31% | Val Loss: 1.7724 | Val Acc: 68.60%
Epoch [78/100] Train Loss: 1.6452 | Train Acc: 81.57% | Val Loss: 1.7324 | Val Acc: 72.94%
Epoch [79/100] Train Loss: 1.6285 | Train Acc: 83.27% | Val Loss: 1.7240 | Val Acc: 73.40%
Epoch [80/100] Train Loss: 1.6224 | Train Acc: 83.96% | Val Loss: 1.7315 | Val Acc: 72.56%
Epoch [81/100] Train Loss: 1.6186 | Train Acc: 84.27% | Val Loss: 1.7255 | Val Acc: 73.44%
Epoch [82/100] Train Loss: 1.6176 | Train Acc: 84.41% | Val Loss: 1.7299 | Val Acc: 73.02%
Epoch [83/100] Train Loss: 1.6154 | Train Acc: 84.60% | Val Loss: 1.7250 | Val Acc: 73.60%
Epoch [84/100] Train Loss: 1.6152 | Train Acc: 84.64% | Val Loss: 1.7243 | Val Acc: 73.36%
Epoch [85/100] Train Loss: 1.6135 | Train Acc: 84.85% | Val Loss: 1.7149 | Val Acc: 74.54%
Epoch [86/100] Train Loss: 1.6077 | Train Acc: 85.36% | Val Loss: 1.7179 | Val Acc: 74.04%
Epoch [87/100] Train Loss: 1.6069 | Train Acc: 85.49% | Val Loss: 1.7206 | Val Acc: 73.80%
Epoch [88/100] Train Loss: 1.6071 | Train Acc: 85.48% | Val Loss: 1.7210 | Val Acc: 73.66%
Epoch [89/100] Train Loss: 1.6054 | Train Acc: 85.68% | Val Loss: 1.7260 | Val Acc: 73.28%
Epoch [90/100] Train Loss: 1.6049 | Train Acc: 85.76% | Val Loss: 1.7244 | Val Acc: 73.22%
Epoch [91/100] Train Loss: 1.6022 | Train Acc: 85.97% | Val Loss: 1.7228 | Val Acc: 73.56%
Epoch [92/100] Train Loss: 1.6022 | Train Acc: 85.94% | Val Loss: 1.7193 | Val Acc: 74.00%
Epoch [93/100] Train Loss: 1.5987 | Train Acc: 86.30% | Val Loss: 1.7168 | Val Acc: 74.26%
Epoch [94/100] Train Loss: 1.5993 | Train Acc: 86.31% | Val Loss: 1.7138 | Val Acc: 74.42%
Epoch [95/100] Train Loss: 1.5982 | Train Acc: 86.36% | Val Loss: 1.7170 | Val Acc: 74.26%
Epoch [96/100] Train Loss: 1.5990 | Train Acc: 86.28% | Val Loss: 1.7206 | Val Acc: 73.72%
Epoch [97/100] Train Loss: 1.5961 | Train Acc: 86.58% | Val Loss: 1.7223 | Val Acc: 73.46%
Epoch [98/100] Train Loss: 1.5951 | Train Acc: 86.71% | Val Loss: 1.7214 | Val Acc: 73.70%
Epoch [99/100] Train Loss: 1.5972 | Train Acc: 86.46% | Val Loss: 1.7168 | Val Acc: 74.18%
Epoch [100/100] Train Loss: 1.5928 | Train Acc: 86.93% | Val Loss: 1.7223 | Val Acc: 73.50%
```

```
# Load the best model and evaluate on the test set
model.eval()

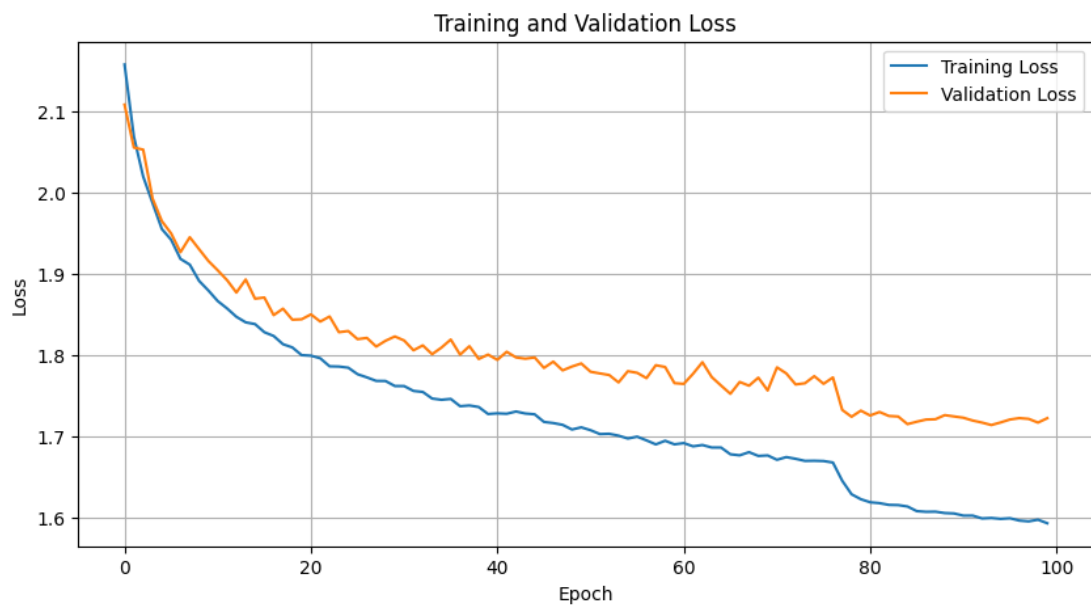
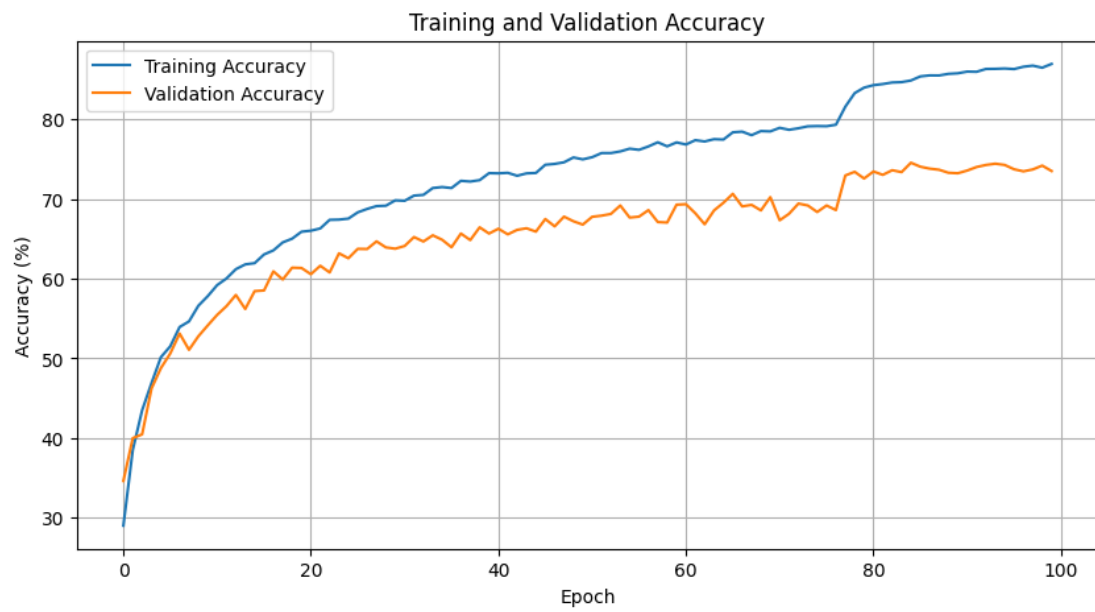
test_correct = 0
test_total = 0
with torch.no_grad():
    for test_inputs, test_labels in testloader:
        test_inputs, test_labels = test_inputs.to(device), test_labels.to(device)
        test_outputs = model(test_inputs)
        _, test_predicted = test_outputs.max(1)
        test_total += test_labels.size(0)
        test_correct += test_predicted.eq(test_labels).sum().item()
test_accuracy = 100. * test_correct / test_total
print(f"Best Model Test Accuracy: {test_accuracy:.2f}%")

Best Model Test Accuracy: 80.19%
```

5.

項目	說明	優點	限制
影像正規化	將像素值统一到特定範圍	穩定訓練，改善收斂速度	當資料已一致時作用有限
資料增強	對訓練樣本施加多樣變換	增強泛化能力，防止過擬合	不當增強可能造成非真實樣本
學習率策略	動態調整訓練中學習率	加快收斂，跳出局部極小點	需要策略調參，選擇影響結果
參數更新策略	權重調整方法	影響訓練穩定性與效率	複雜方法可能增加計算負擔

6.



Task3

lab1-1 遇到的問題

1. MLP hidden layer 設計層數太少，導致準確率太低，故增加層數來改善(1 層增至 5 層)
2. 在訓練過程中，出現驗證損失比訓練損失高，嘗試減少 MLP 之 hidden layer，嘗試改成 5 層改成 2 層且減少 MLP 每層神經元個數，避免模型過於複雜造成 overfitting
3. 遇到報錯訊息，除了自行上網找資料外也有求助 AI 詢問
4. 詢問 AI 關於此基礎架構 MLP 用於辨識 MNIST 的準確率極限(約 94%-96%)

lab1-2 遇到的問題

1. 訓練 50 個 Epochs 後損失不降，詢問 AI 如何調整，決定調整 lr 從 0.01 改成 0.001，並將 augmentation 從 weak 改成 strong 且為了妥善利用 GPU 平行能力
2. 後續第二次訓練在 EPOCH 50，VAL_ACC 卡在 65%上下，Loss 卡在 1.8 左右。詢問 AI 如何處理，他加入了學習率排程器做為調整及改善及加入 SGD+Momentum+weight decay，並中斷後再重新訓練，在第三次訓練中，Acc 及 Loss 明顯改善許多